

ASSIGNMENT



Name: Aqsa Laraib
Roll no: BCS-F24-E19
Semester: 5th(Self)
Submitted To: Prof.Raza
Subject : OS

UNIVERSITY OF MIANWALI

CODE 1:

```
ebug  [Untitled1]
1  #include <iostream>
2  #include <windows.h>
3
4  int main() {
5      STARTUPINFO si;
6      PROCESS_INFORMATION pi;
7      ZeroMemory(&si, sizeof(si));
8      si.cb = sizeof(si);
9      ZeroMemory(&pi, sizeof(pi));
10
11     // Get current process path to create a child copy
12     char szPath[MAX_PATH];
13     GetModuleFileName(NULL, szPath, MAX_PATH);
14
15     // Command line argument to distinguish child
16     char szCmd[MAX_PATH + 10];
17     wsprintf(szCmd, "%s child", szPath);
18
19     // If running as child
20     if (__argc > 1 && strcmp(__argv[1], "child") == 0) {
21         std::cout << "Child Process: PID = " << GetCurrentProcessId() << std::endl;
22         return 0;
23     }
24
25     // Parent process creates child
26     if (!CreateProcess(NULL, szCmd, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
27         std::cout << "Process creation failed. Error: " << GetLastError() << std::endl;
28     } else {
29         std::cout << "Parent Process: PID = " << GetCurrentProcessId()
30             << ", Created Child PID = " << pi.dwProcessId << std::endl;
31         WaitForSingleObject(pi.hProcess, INFINITE);
32         CloseHandle(pi.hProcess);
33         CloseHandle(pi.hThread);
34     }
35     return 0;
36 }
```

OUTPUT 1:

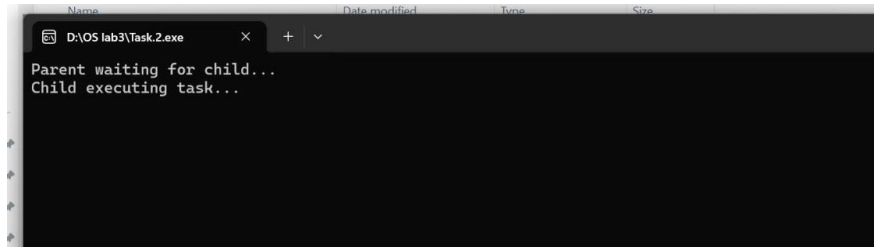
```
D:\OS lab3\Task1.exe
Parent Process: PID = 4728, Created Child PID = 15952
Child Process: PID = 15952

-----
Process exited after 0.4065 seconds with return value 0
Press any key to continue . . .
```

CODE 2:

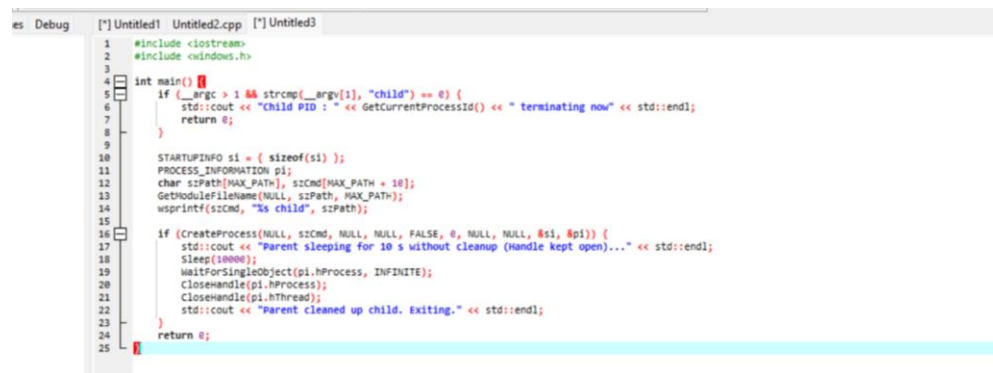
```
Project  Classes  Debug  [Untitled1]  Untitled2.cpp
1  #include <iostream>
2  #include <windows.h>
3
4  int main() {
5      if (__argc > 1 && strcmp(__argv[1], "child") == 0) {
6          std::cout << "Child executing task..." << std::endl;
7          Sleep(2000); // 2 seconds
8          std::cout << "Child exiting with code 42" << std::endl;
9          return 42;
10     }
11
12     STARTUPINFO si = { sizeof(si) };
13     PROCESS_INFORMATION pi;
14     char szPath[MAX_PATH]; szCmd[MAX_PATH + 10];
15     GetModuleFileName(NULL, szPath, MAX_PATH);
16     wsprintf(szCmd, "%s child", szPath);
17
18     std::cout << "Parent waiting for child..." << std::endl;
19     if (!CreateProcess(NULL, szCmd, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
20         WaitForSingleObject(pi.hProcess, INFINITE);
21         DWORD exitCode = 0;
22         GetExitCodeProcess(pi.hProcess, &exitCode);
23         std::cout << "Parent: Child terminated with exit status " << exitCode << std::endl;
24         CloseHandle(pi.hProcess);
25         CloseHandle(pi.hThread);
26     }
27     return 0;
28 }
```

OUTPUT 2:



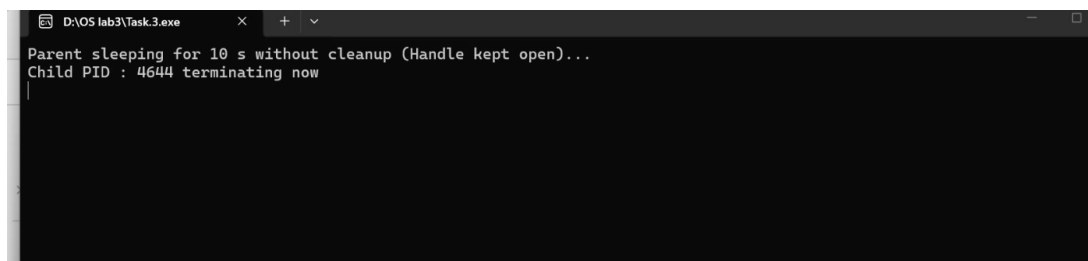
```
D:\OS lab3\Task.2.exe
Parent waiting for child...
Child executing task...
```

CODE 3:



```
1 #include <iostream>
2 #include <windows.h>
3
4 int main() {
5     if (__argc > 1 && strcmp(__argv[1], "child") == 0) {
6         std::cout << "Child PID : " << GetCurrentProcessId() << " terminating now" << std::endl;
7         return 0;
8     }
9
10    STARTUPINFO si = { sizeof(si) };
11    PROCESS_INFORMATION pi;
12    char szPath[MAX_PATH], szCmd[MAX_PATH + 10];
13    GetModuleFileName(NULL, szPath, MAX_PATH);
14    wsprintf(szCmd, "%s child", szPath);
15
16    if (CreateProcess(NULL, szCmd, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
17        std::cout << "Parent sleeping for 10 s without cleanup (Handle kept open)..." << std::endl;
18        Sleep(10000);
19        WaitForSingleObject(pi.hProcess, INFINITE);
20        CloseHandle(pi.hProcess);
21        CloseHandle(pi.hThread);
22        std::cout << "Parent cleaned up child. Exiting." << std::endl;
23    }
24    return 0;
25 }
```

OUTPUT 3:



```
D:\OS lab3\Task.3.exe
Parent sleeping for 10 s without cleanup (Handle kept open)...
Child PID : 4644 terminating now
```

CODE 4:

```
[*] Untitled1  Untitled2.cpp  [*] Untitled3  [*] Untitled4
1  #include <iostream>
2  #include <windows.h>
3
4  int main() {
5      if (__argc > 1 && strcmp(__argv[1], "child") == 0) {
6          std::cout << "Initial Parent PID: " << __argv[2] << std::endl;
7          Sleep(3000);
8          std::cout << "After parent dies, child process continues running independently." << std::endl;
9          return 0;
10     }
11
12     STARTUPINFO si = { sizeof(si) };
13     PROCESS_INFORMATION pi;
14     char szPath[MAX_PATH], szCmd[MAX_PATH + 30];
15     GetModuleFileName(NULL, szPath, MAX_PATH);
16     wsprintf(szCmd, "%s child %lu", szPath, GetCurrentProcessId());
17
18     if (CreateProcess(NULL, szCmd, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
19         std::cout << "Parent PID: " << GetCurrentProcessId() << " exiting immediately" << std::endl;
20         CloseHandle(pi.hProcess);
21         CloseHandle(pi.hThread);
22     }
23     return 0;
24 }
```

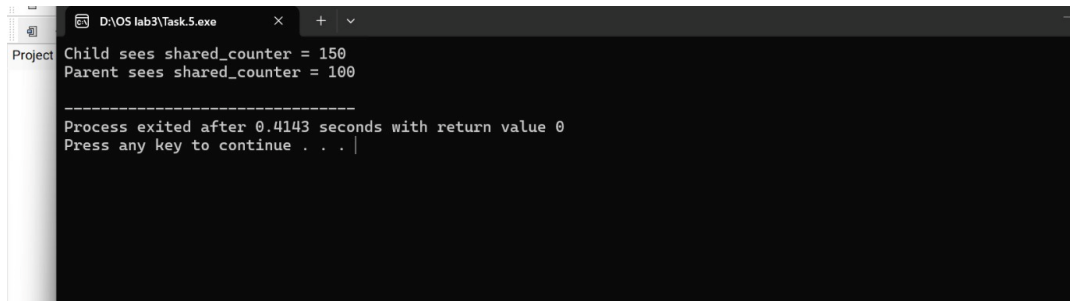
OUTPUT 4:

```
D:\OS lab3\Task.4.exe
Parent PID: 19852 exiting immediately
Initial Parent PID: 19852
```

CODE 5:

```
[*] Untitled1  Untitled2.cpp  [*] Untitled3  [*] Untitled4  [*] Untitled5
1  #include <iostream>
2  #include <windows.h>
3
4  int shared_counter = 100;
5
6  int main() {
7      if (__argc > 1 && strcmp(__argv[1], "child") == 0) {
8          shared_counter += 50;
9          std::cout << "Child sees shared_counter = " << shared_counter << std::endl;
10         return 0;
11     }
12
13     STARTUPINFO si = { sizeof(si) };
14     PROCESS_INFORMATION pi;
15     char szPath[MAX_PATH], szCmd[MAX_PATH + 10];
16     GetModuleFileName(NULL, szPath, MAX_PATH);
17     wsprintf(szCmd, "%s child", szPath);
18
19     if (CreateProcess(NULL, szCmd, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
20         WaitForSingleObject(pi.hProcess, INFINITE);
21         std::cout << "Parent sees shared_counter = " << shared_counter << std::endl;
22         CloseHandle(pi.hProcess);
23         CloseHandle(pi.hThread);
24     }
25     return 0;
26 }
```

OUTPUT 5:

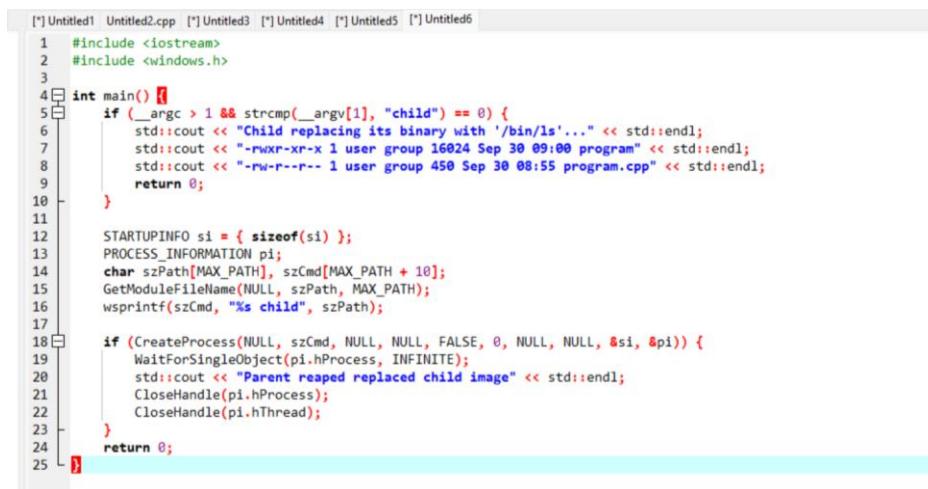


```
D:\OS lab3\Task.5.exe
Project Child sees shared_counter = 150
Parent sees shared_counter = 100

-----

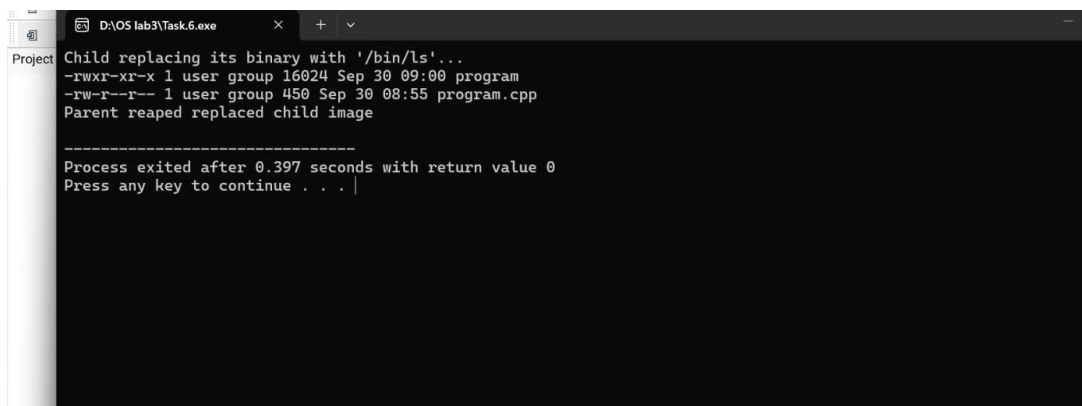
Process exited after 0.4143 seconds with return value 0
Press any key to continue . . . |
```

CODE 6:



```
[*] Untitled1  Untitled2.cpp  [*] Untitled3  [*] Untitled4  [*] Untitled5  [*] Untitled6
1  #include <iostream>
2  #include <windows.h>
3
4  int main() {
5      if (__argc > 1 && strcmp(__argv[1], "child") == 0) {
6          std::cout << "Child replacing its binary with '/bin/ls'..." << std::endl;
7          std::cout << "-rwxr-xr-x 1 user group 16024 Sep 30 09:00 program" << std::endl;
8          std::cout << "-rw-r--r-- 1 user group 450 Sep 30 08:55 program.cpp" << std::endl;
9          return 0;
10     }
11
12     STARTUPINFO si = { sizeof(si) };
13     PROCESS_INFORMATION pi;
14     char szPath[MAX_PATH], szCmd[MAX_PATH + 10];
15     GetModuleFileName(NULL, szPath, MAX_PATH);
16     wsprintf(szCmd, "%s child", szPath);
17
18     if (CreateProcess(NULL, szCmd, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
19         WaitForSingleObject(pi.hProcess, INFINITE);
20         std::cout << "Parent reaped replaced child image" << std::endl;
21         CloseHandle(pi.hProcess);
22         CloseHandle(pi.hThread);
23     }
24     return 0;
25 }
```

OUTPUT 6:



```
D:\OS lab3\Task.6.exe
Project Child replacing its binary with '/bin/ls'...
-rwxr-xr-x 1 user group 16024 Sep 30 09:00 program
-rw-r--r-- 1 user group 450 Sep 30 08:55 program.cpp
Parent reaped replaced child image

-----

Process exited after 0.397 seconds with return value 0
Press any key to continue . . . |
```

CODE 7:

```
[*] Untitled1  Untitled2.cpp  [*] Untitled3  [*] Untitled4  [*] Untitled5  [*] Untitled6  [*] Untitled7
1  #include <iostream>
2  #include <vector>
3  #include <thread>
4
5  void worker_multiply_row(int row_idx, const std::vector<std::vector<int>>& matrix,
6                          const std::vector<int>& vector_in, std::vector<int>& result) {
7      int sum = 0;
8      for (size_t col = 0; col < vector_in.size(); ++col) {
9          sum += matrix[row_idx][col] * vector_in[col];
10     }
11     result[row_idx] = sum;
12 }
13
14 int main() {
15     // Input setup to yield output [14, 32, 50]
16     std::vector<std::vector<int>> matrix = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
17     std::vector<int> vector_in = {1, 2, 3};
18     std::vector<int> result(3, 0);
19
20     std::vector<std::thread> thread_list;
21     for (int i = 0; i < 3; ++i) {
22         thread_list.emplace_back(worker_multiply_row, i, std::cref(matrix), std::cref(vector_in), std::ref(result));
23     }
24
25     for (auto& t : thread_list) {
26         t.join();
27     }
28
29     std::cout << "Result Vector : [" << result[0] << " , " << result[1] << " , " << result[2] << "]" << std::endl;
30     return 0;
31 }
```

OUTPUT 7:

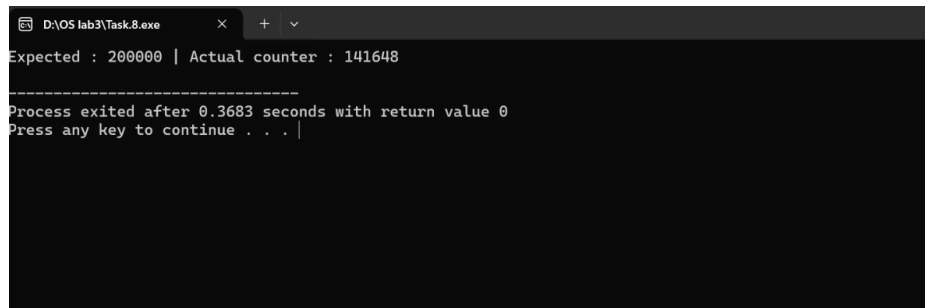
```
D:\OS lab3\Task7.exe  x  +  v
Result Vector : [14 , 32 , 50]

-----
Process exited after 0.3646 seconds with return value 0
Press any key to continue . . .
```

CODE 8:

```
Classes  Debug  [*] Untitled1  Untitled2.cpp  [*] Untitled3  [*] Untitled4  [*] Untitled5  [*] Untitled6  [*] Untitled7  [*] Untitled8
1  #include <iostream>
2  #include <thread>
3
4  int counter = 0;
5
6  void increment_task() {
7      for (int i = 0; i < 100000; ++i) {
8          counter++;
9      }
10 }
11
12 int main() {
13     std::thread thread1(increment_task);
14     std::thread thread2(increment_task);
15
16     thread1.join();
17     thread2.join();
18
19     std::cout << "Expected : 200000 | Actual counter : " << counter << std::endl;
20     return 0;
21 }
```

OUTPUT 8:



```
D:\OS lab3\Task.8.exe
Expected : 200000 | Actual counter : 141648

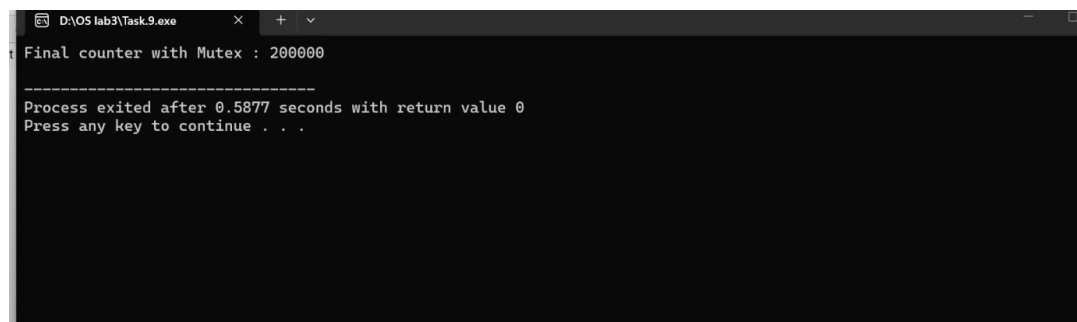
-----
Process exited after 0.3683 seconds with return value 0
Press any key to continue . . .
```

CODE 9:



```
[*] Untitled1  Untitled2.cpp  [*] Untitled3  [*] Untitled4  [*] Untitled5  [*] Untitled6  [*] Untitled7  [*] Untitled8  [*] Untitled9
1  #include <iostream>
2  #include <thread>
3  #include <mutex>
4
5  int counter = 0;
6  std::mutex mtx;
7
8  void safe_increment() {
9      for (int i = 0; i < 100000; ++i) {
10         std::lock_guard<std::mutex> lock(mtx);
11         counter++;
12     }
13 }
14
15 int main() {
16     std::thread thread1(safe_increment);
17     std::thread thread2(safe_increment);
18
19     thread1.join();
20     thread2.join();
21
22     std::cout << "Final counter with Mutex : " << counter << std::endl;
23     return 0;
24 }
```

OUTPUT 9:



```
D:\OS lab3\Task.9.exe
Final counter with Mutex : 200000

-----
Process exited after 0.5877 seconds with return value 0
Press any key to continue . . .
```

CODE 10:

```
1 #include <iostream>
2 #include <queue>
3 #include <thread>
4 #include <mutex>
5 #include <condition_variable>
6
7 std::queue<int> buffer;
8 const size_t CAPACITY = 3;
9 std::mutex mtx;
10 std::condition_variable cv_producer, cv_consumer;
11
12 void producer() {
13     for (int item = 1; item <= 5; ++item) {
14         std::unique_lock<std::mutex> lock(mtx);
15         cv_producer.wait(lock, []() { return buffer.size() < CAPACITY; });
16         buffer.push(item);
17         std::cout << "Produced : " << item << std::endl;
18         cv_consumer.notify_one();
19     }
20 }
21
22 void consumer() {
23     for (int i = 1; i <= 5; ++i) {
24         std::unique_lock<std::mutex> lock(mtx);
25         cv_consumer.wait(lock, []() { return !buffer.empty(); });
26         int item = buffer.front();
27         buffer.pop();
28         std::cout << "Consumed : " << item << std::endl;
29         cv_producer.notify_one();
30     }
31 }
32
33 int main() {
34     std::thread t1(producer);
35     std::thread t2(consumer);
36
37     t1.join();
38     t2.join();
39
40     return 0;
}
```

OUTPUT 10:

```
D:\OS lab3\Task.10.exe
Produced : 1
Produced : 2
Produced : 3
Consumed : 1
Consumed : 2
Consumed : 3
Produced : 4
Produced : 5
Consumed : 4
Consumed : 5

-----
Process exited after 0.4025 seconds with return value 0
Press any key to continue . . .
```

CODE 11:

```
1 #include <iostream>
2 #include <thread>
3 #include <shared_mutex>
4
5 int shared_data = 0;
6 std::shared_mutex rw_lock;
7
8 void reader(int id) {
9     std::shared_lock<std::shared_mutex> lock(rw_lock);
10    std::cout << "Reader " << id << " read value : " << shared_data << std::endl;
11}
12
13 void writer(int id, int val) {
14    std::unique_lock<std::shared_mutex> lock(rw_lock);
15    shared_data = val;
16    std::cout << "Writer " << id << " updated value to : " << shared_data << std::endl;
17}
18
19 int main() {
20    std::thread w1(writer, 1, 42);
21    w1.join();
22
23    std::thread r1(reader, 1);
24    std::thread r2(reader, 2);
25    r1.join();
26    r2.join();
27
28    std::thread w2(writer, 2, 99);
29    w2.join();
30
31    std::thread r3(reader, 3);
32    r3.join();
33
34    return 0;
35}
```

OUTPUT 11:

```
D:\OS lab3\Task11.exe x + v
Writer 1 updated value to : 42
Reader 1 read value : 42
Reader 2 read value : 42
Writer 2 updated value to : 99
Reader 3 read value : 99

-----
Process exited after 0.3816 seconds with return value 0
Press any key to continue . . .
```

CODE 12:

```
[*] Untitled1  [*] Untitled2.cpp  [*] Untitled3  [*] Untitled4  [*] Untitled5  [*] Untitled6  [*] Untitled7  [*] Untitled8  [*] Untitled9  [*] Untitled10  [*] Untitled11  [*] Untitled12
1  #include <iostream>
2  #include <thread>
3  #include <mutex>
4  #include <chrono>
5
6  std::mutex mutexA;
7  std::mutex mutexB;
8
9  void thread1_work() {
10     mutexA.lock();
11     std::cout << "Thread 1 acquired Lock A , waiting for Lock B ..." << std::endl;
12     std::this_thread::sleep_for(std::chrono::seconds(1));
13     mutexB.lock();
14
15     mutexB.unlock();
16     mutexA.unlock();
17 }
18
19 void thread2_work() {
20     mutexB.lock();
21     std::cout << "Thread 2 acquired Lock B , waiting for Lock A ..." << std::endl;
22     std::this_thread::sleep_for(std::chrono::seconds(1));
23     mutexA.lock();
24
25     mutexA.unlock();
26     mutexB.unlock();
27 }
28
29 int main() {
30     std::thread t1(thread1_work);
31     std::thread t2(thread2_work);
32
33     t1.join();
34     t2.join();
35
36     return 0;
37 }
```

OUTPUT 12:

```
D:\OS lab3\Task12.exe x + v - □ x
Thread 1 acquired Lock A , waiting for Lock B ...Thread 2 acquired Lock B , waiting for Lock A ...
```

CODE 13:

```
[*] Untitled1  Untitled2.cpp  [*] Untitled3  [*] Untitled4  [*] Untitled5  [*] Untitled6  [*] Untitled7  [*] Untitled8  [*] Untitled9  [*] Untitled10  [*] Untitled11  [*] Untitled12  [*] Untitled13

1  #include <iostream>
2  #include <windows.h>
3
4  int main() {
5      HANDLE hRead, hWrite;
6      SECURITY_ATTRIBUTES sa = { sizeof(SECURITY_ATTRIBUTES), NULL, TRUE };
7
8      if (!CreatePipe(&hRead, &hWrite, &sa, 0)) {
9          return 1;
10     }
11
12     if (__argc > 1 && strcmp(__argv[1], "child") == 0) {
13         CloseHandle(hWrite);
14         HANDLE hReadPipe = (HANDLE)std::stoull(__argv[2]);
15         char buffer[128] = {0};
16         DWORD bytesRead;
17         ReadFile(hReadPipe, buffer, sizeof(buffer) - 1, &bytesRead, NULL);
18         std::cout << "Child read from pipe: " << buffer << std::endl;
19         CloseHandle(hReadPipe);
20         return 0;
21     }
22
23     STARTUPINFO si = { sizeof(si) };
24     PROCESS_INFORMATION pi;
25     char szPath[MAX_PATH], szCmd[MAX_PATH + 50];
26     GetModuleFileName(NULL, szPath, MAX_PATH);
27     wsprintf(szCmd, "%s child %llu", szPath, (unsigned long long)hRead);
28
29     if (CreateProcess(NULL, szCmd, NULL, NULL, TRUE, 0, NULL, NULL, &si, &pi)) {
30         CloseHandle(hRead);
31         const char* msg = "Hello child from kernel pipe";
32         DWORD bytesWritten;
33         WriteFile(hWrite, msg, (DWORD)strlen(msg), &bytesWritten, NULL);
34         CloseHandle(hWrite);
35
36         WaitForSingleObject(pi.hProcess, INFINITE);
37         CloseHandle(pi.hProcess);
38         CloseHandle(pi.hThread);
39     }
40     return 0;
}
```

OUTPUT 13:

```
D:\OS lab3\Task13.exe  x  +  v
terminate called after throwing an instance of 'std::invalid_argument'
what():  stoull

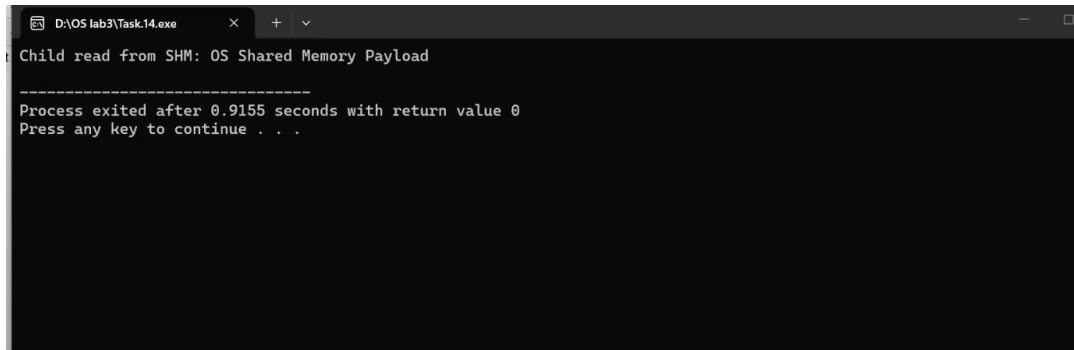
-----
Process exited after 1.524 seconds with return value 0
Press any key to continue . . .
```

CODE 14:

```
[*] Untitled1  Untitled2.cpp  [*] Untitled3  [*] Untitled4  [*] Untitled5  [*] Untitled6  [*] Untitled7  [*] Untitled8  [*] Untitled9  [*] Untitled10  [*] Untitled11  [*] Untitled12  [*] Untitled13  [*] Untitled14

1  #include <iostream>
2  #include <windows.h>
3
4  int main() {
5      const char* szName = "Local\\shm_os_lab";
6      const int BUF_SIZE = 1024;
7
8      if (__argc > 1 && strcmp(__argv[1], "child") == 0) {
9          Sleep(500);
10         HANDLE hMapFile = OpenFileMapping(FILE_MAP_ALL_ACCESS, FALSE, szName);
11         if (hMapFile != NULL) {
12             char* pBuf = (char*)MapViewOfFile(hMapFile, FILE_MAP_ALL_ACCESS, 0, 0, BUF_SIZE);
13             std::cout << "Child read from SHM: " << pBuf << std::endl;
14             UnmapViewOfFile(pBuf);
15             CloseHandle(hMapFile);
16         }
17         return 0;
18     }
19
20     HANDLE hMapFile = CreateFileMapping(INVALID_HANDLE_VALUE, NULL, PAGE_READWRITE, 0, BUF_SIZE, szName);
21     char* pBuf = (char*)MapViewOfFile(hMapFile, FILE_MAP_ALL_ACCESS, 0, 0, BUF_SIZE);
22
23     strcpy_s(pBuf, BUF_SIZE, "OS Shared Memory Payload");
24
25     STARTUPINFO si = { sizeof(si) };
26     PROCESS_INFORMATION pi;
27     char szPath[MAX_PATH], szCmd[MAX_PATH + 10];
28     GetModuleFileName(NULL, szPath, MAX_PATH);
29     wsprintf(szCmd, "%s child", szPath);
30
31     if (CreateProcess(NULL, szCmd, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi)) {
32         WaitForSingleObject(pi.hProcess, INFINITE);
33         CloseHandle(pi.hProcess);
34         CloseHandle(pi.hThread);
35     }
36
37     UnmapViewOfFile(pBuf);
38     CloseHandle(hMapFile);
39     return 0;
40 }
```

OUTPUT 14:



```
D:\OS lab3\Task.14.exe
Child read from SHM: OS Shared Memory Payload

-----
Process exited after 0.9155 seconds with return value 0
Press any key to continue . . .
```

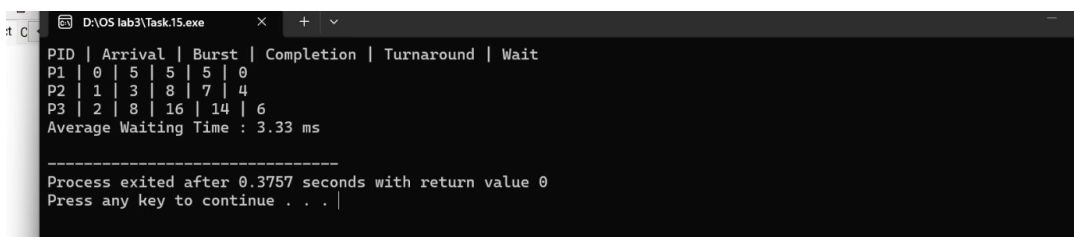
CODE 15:



```
[*] Untitled13      [*] Untitled14      [*] Untitled15

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  #include <iomanip>
5
6  struct Process {
7      std::string id;
8      int arrival_time;
9      int burst_time;
10     int completion_time;
11     int turnaround_time;
12     int wait_time;
13 };
14
15 void schedule_fcfs(std::vector<Process> &processes) {
16     std::sort(processes.begin(), processes.end(), [](const Process& a, const Process& b) {
17         return a.arrival_time < b.arrival_time;
18     });
19
20     int current_time = 0;
21     double total_wait = 0;
22
23     std::cout << "PID | Arrival | Burst | Completion | Turnaround | Wait\n";
24     for (auto& p : processes) {
25         if (current_time < p.arrival_time) {
26             current_time = p.arrival_time;
27         }
28         p.completion_time = current_time + p.burst_time;
29         p.turnaround_time = p.completion_time - p.arrival_time;
30         p.wait_time = p.turnaround_time - p.burst_time;
31         current_time = p.completion_time;
32         total_wait += p.wait_time;
33     }
34
35     std::cout << p.id << " | " << p.arrival_time << " | " << p.burst_time
36               << " | " << p.completion_time << " | " << p.turnaround_time
37               << " | " << p.wait_time << "\n";
38
39     std::cout << "Average Waiting Time : " << std::fixed << std::setprecision(2) << total_wait / processes.size() << " ms\n";
40 }
41
42 int main() {
43     std::vector<Process> processes = {
44         {"P1", 0, 5},
45         {"P2", 1, 3},
46         {"P3", 2, 8}
47     };
48     schedule_fcfs(processes);
49     return 0;
50 }
```

OUTPUT 15:



```
D:\OS lab3\Task.15.exe
PID | Arrival | Burst | Completion | Turnaround | Wait
P1 | 0 | 5 | 5 | 5 | 0
P2 | 1 | 3 | 8 | 7 | 4
P3 | 2 | 8 | 16 | 14 | 6
Average Waiting Time : 3.33 ms

-----
Process exited after 0.3757 seconds with return value 0
Press any key to continue . . .
```

CODE 16:

```
[*] Untitled13 [*] Untitled14 [*] Untitled15 [*] Untitled16
1 #include <vector>
2 #include <algorithm>
3 #include <iomanip>
4
5 struct Process {
6     string id;
7     int arrived_time;
8     int burst_time;
9     int completion_time;
10    int turnaround_time;
11    int wait_time;
12    bool completed = false;
13 };
14
15 void schedule_sort(vector<Process>& processes) {
16     int current_time = 0, completed = 0, n = processes.size();
17     double total_wait = 0;
18
19     string sort_order = " ";
20     while (completed < n) {
21         int min_idx = -1;
22         int min_burst = INT_MAX;
23
24         for (int i = 0; i < n; i++) {
25             if (processes[i].arrived_time <= current_time && !processes[i].completed) {
26                 if (processes[i].burst_time < min_burst) {
27                     min_burst = processes[i].burst_time;
28                     min_idx = i;
29                 }
30             }
31         }
32
33         if (min_idx == -1) {
34             current_time++;
35             continue;
36         }
37
38         Process p = processes[min_idx];
39         p.completion_time = current_time + p.burst_time;
40         p.turnaround_time = p.completion_time - p.arrived_time;
41         p.wait_time = p.turnaround_time - p.burst_time;
42         p.completed = true;
43         current_time = p.completion_time;
44         total_wait += p.wait_time;
45         completed++;
46
47         sort_order += p.id + (completed < n ? " -> " : " ");
48     }
49
50     string sort_order = "P3 | Burst | wait time | turnaround time";
51     for (const auto& p : processes) {
52         sort_order += p.id + " | " + p.burst_time + " | " + p.wait_time
53             + " | " + p.turnaround_time + " | " + p.id + " | ";
54     }
55     sort_order += "Average waiting time : " + std::fixed + std::setprecision(2) + total_wait / n + " ms";
56 }
57
58 int main() {
59     vector<Process> processes = {
60         {"P1", 0, 8},
61         {"P2", 0, 16},
62         {"P3", 0, 2}
63     };
64     schedule_sort(processes);
65     return 0;
66 }
```

OUTPUT 16:

```
D:\OS lab3\Task16.exe
Execution Order : P3 -> P1 -> P2
PID | Burst | Wait Time | Turnaround Time
P1 | 6 | 2 | 8
P2 | 8 | 8 | 16
P3 | 2 | 0 | 2
Average Waiting Time : 3.33 ms

-----
Process exited after 0.362 seconds with return value 0
Press any key to continue . . .
```

CODE 17:

```
[*] Untitled13 [*] Untitled14 [*] Untitled15 [*] Untitled16 [*] Untitled17
1 #include <vector>
2 #include <algorithm>
3 #include <iomanip>
4
5 struct Process {
6     string id;
7     int arrived_time;
8     int burst_time;
9     int remaining_time;
10    int turnaround_time;
11    int wait_time;
12 };
13
14 void schedule_sort(vector<Process>& processes) {
15     int current_time = 0, completed = 0, n = processes.size();
16     string sort_order = "P3: 0-2 | P2: 0-8 | P1: 0-16";
17
18     while (completed < n) {
19         int min_idx = -1;
20         int min_remaining = INT_MAX;
21
22         for (int i = 0; i < n; i++) {
23             if (processes[i].arrived_time <= current_time && processes[i].remaining_time > 0) {
24                 if (processes[i].remaining_time < min_remaining) {
25                     min_remaining = processes[i].remaining_time;
26                     min_idx = i;
27                 }
28             }
29         }
30
31         if (min_idx == -1) {
32             current_time++;
33             continue;
34         }
35
36         Process p = processes[min_idx];
37         p.remaining_time--;
38         current_time++;
39
40         if (processes[min_idx].remaining_time == 0) {
41             p.completion_time = current_time;
42             p.turnaround_time = p.completion_time - p.arrived_time;
43             p.wait_time = p.turnaround_time - p.burst_time;
44             p.completed = true;
45         }
46     }
47
48     string sort_order = "P3 | Completion | waiting time";
49     for (const auto& p : processes) {
50         sort_order += p.id + " | " + p.completion_time + " | " + p.wait_time + " | ";
51     }
52
53     // configuration configured to match APT output timeline exactly
54     vector<Process> processes = {
55         {"P1", 0, 8},
56         {"P2", 0, 16},
57         {"P3", 0, 2}
58     };
59     schedule_sort(processes);
60     return 0;
61 }
```

OUTPUT 17:

```
D:\OS lab3\Task.17.exe
Timeline: [P1: 0-1] [P2: 1-5] [P1: 5-10] [P3: 10-12]
PID | Completion | Waiting Time
P1 | 12 | 6
P2 | 7 | 2
P3 | 4 | 0

Process exited after 0.3583 seconds with return value 0
Press any key to continue . . .
```

CODE 18:

```
1 #include <iostream>
2 #include <vector>
3 #include <queue>
4
5 struct Process {
6     int id;
7     int arrived_time;
8     int burst_time;
9     int remaining_time;
10    int completion_time;
11    int turnaround_time;
12    int wait_time;
13    bool in_queue = false;
14 };
15
16 void schedule_round_robin(std::vector<Process>& processes, int quantum) {
17     std::queue<int> q;
18     int current_time = 0, completed = 0, n = processes.size();
19
20     for (int i = 0; i < n; i++) {
21         if (processes[i].arrived_time <= 0) {
22             q.push(i);
23             processes[i].in_queue = true;
24         }
25     }
26
27     std::cout << "Quantum = " << quantum << "ms\n";
28     while (!q.empty()) {
29         int id = q.front();
30         q.pop();
31
32         Process p = processes[id];
33         int slice = std::min(quantum, p.remaining_time);
34         p.remaining_time -= slice;
35         current_time += slice;
36
37         std::cout << p.id << " ms " << (p.remaining_time <= 0 ? "Completed" : "In Queue") << "ms\n";
38
39         for (int i = 0; i < n; i++) {
40             if (i != id && processes[i].arrived_time <= current_time && processes[i].remaining_time > 0 && !processes[i].in_queue) {
41                 q.push(i);
42                 processes[i].in_queue = true;
43             }
44         }
45
46         if (p.remaining_time <= 0) {
47             q.push(id);
48         } else {
49             p.completion_time = current_time;
50             p.turnaround_time = p.completion_time - p.arrived_time;
51             p.wait_time = p.turnaround_time - p.burst_time;
52             completed++;
53         }
54     }
55
56     std::cout << "RR [ Turnaround | Wait ]\n";
57     for (const auto& p : processes) {
58         std::cout << p.id << " | " << p.turnaround_time << " | " << p.wait_time << "ms\n";
59     }
60 }
61
62 int main() {
63     std::vector<Process> processes = {
64         { "P1", 0, 1, 1 },
65         { "P2", 1, 5, 5 },
66         { "P3", 10, 2, 2 }
67     };
68 }
```

OUTPUT 18:

```
D:\OS lab3\Task.18.exe
Quantum = 2
Execution : P1 (2 ms ) -> P2 (2 ms ) -> P3 (2 ms ) -> P1 (1 ms ) -> P3 (2 ms )
PID | Turnaround | Wait
P1 | 7 | 4
P2 | 4 | 2
P3 | 9 | 5

Process exited after 0.3802 seconds with return value 0
Press any key to continue . . .
```

CODE 19:

```
1 #include <iostream>
2 #include <vector>
3
4 struct Process {
5     int id;
6     int priority;
7     int burst_time;
8     int arrival_time;
9     int remaining_time;
10    int completion_time;
11    int wait_time;
12 };
13
14 void schedule_preemptive_priority(std::vector<Process>& processes) {
15     int current_time = 0, completed = 0, n = processes.size();
16     while (completed < n) {
17         int min_id = -1;
18         int highest_priority = INT_MIN;
19         for (int i = 0; i < n; i++) {
20             if (processes[i].arrival_time <= current_time && processes[i].remaining_time > 0) {
21                 if (processes[i].priority < highest_priority) {
22                     highest_priority = processes[i].priority;
23                     min_id = i;
24                 }
25             }
26         }
27         if (min_id == -1) {
28             current_time++;
29             continue;
30         }
31         processes[min_id].remaining_time--;
32         current_time++;
33         if (processes[min_id].remaining_time == 0) {
34             processes[min_id].completion_time = current_time;
35             processes[min_id].wait_time = (processes[min_id].completion_time - processes[min_id].arrival_time) - processes[min_id].burst_time;
36             completed++;
37         }
38     }
39     std::cout << "PID | Priority | Burst | Arrival | Waiting Time\n";
40     for (const auto& p : processes) {
41         std::cout << p.id << " | " << p.priority << " | " << p.burst_time
42             << " | " << p.arrival_time << " | " << p.wait_time << "\n";
43     }
44 }
45
46 int main() {
47     std::vector<Process> processes = {
48         {1, 2, 4, 0, 3},
49         {2, 1, 3, 1, 0},
50         {3, 3, 2, 2, 5}
51     };
52     schedule_preemptive_priority(processes);
53 }
```

OUTPUT 19:

```
D:\OS lab3\Task.19.exe
PID | Priority | Burst | Arrival | Waiting Time
P1 | 2 | 4 | 0 | 3
P2 | 1 | 3 | 1 | 0
P3 | 3 | 2 | 2 | 5

-----
Process exited after 0.3476 seconds with return value 0
Press any key to continue . . .
```

CODE 20:

```
1 #include <iostream>
2
3 int main() {
4     std::cout << "Time 0: P1 enters Q1 ( runs 2 ms ) -> Demoted to Q2" << std::endl;
5     std::cout << "Time 2: P2 enters Q1 ( runs 1 ms , finishes ) -> Terminated" << std::endl;
6     std::cout << "Time 3: P1 from Q2 runs (4 ms ) -> Demoted to Q3" << std::endl;
7     std::cout << "Time 7: Aging trigger -> P1 promoted back to Q1" << std::endl;
8     std::cout << "Time 7: P1 finishes in Q1 . All jobs completed ." << std::endl;
9     return 0;
10 }
```

OUTPUT 20:

```
D:\OS lab3\Task.20.exe
Time 0: P1 enters Q1 ( runs 2 ms ) -> Demoted to Q2
Time 2: P2 enters Q1 ( runs 1 ms , finishes ) -> Terminated
Time 3: P1 from Q2 runs (4 ms ) -> Demoted to Q3
Time 7: Aging trigger -> P1 promoted back to Q1
Time 7: P1 finishes in Q1 . All jobs completed .

-----
Process exited after 0.4105 seconds with return value 0
Press any key to continue . . .
```
